

TDTU

Thủ tục:

NGHIÊN CỨU KHOA HỌC SINH VIÊN

Mã số:

TT/P.QLPTKHCN/13/BM18

Ban hành lần: 06

Ngày hiệu lực:

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

TDTU

Công trình Nghiên cứu khoa học sinh viên năm học 2025 - 2026

ỨNG DỤNG HỌC TĂNG CƯỜNG
TRONG TỐI ƯU THUẬT TOÁN ALNS
CHO BÀI TOÁN ĐỊNH TUYẾN XE CÓ
RÀNG BUỘC KHUNG THỜI GIAN

ĐƠN VỊ: KHOA CÔNG NGHỆ THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN:

TIẾN SĨ HỒ THỊ LINH

SINH VIÊN THỰC HIỆN:

HUYỀN NHẬT HUY — MSSV: 523c0012

TP. Hồ Chí Minh, tháng 5 năm 2026

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	---	--

TÓM TẮT CÔNG TRÌNH

Bài toán định tuyến phương tiện có khung thời gian (Vehicle Routing Problem with Time Windows – VRPTW) là một bài toán tối ưu hóa tổ hợp thuộc lớp NP-khó, có ý nghĩa quan trọng trong quản trị vận tải và logistics đô thị. Công trình này nghiên cứu hướng tiếp cận lai DDQN-ALNS, kết hợp Tìm kiếm lân cận lớn thích ứng (Adaptive Large Neighborhood Search – ALNS) với Học tăng cường sâu (Deep Reinforcement Learning – DRL), trong đó sử dụng kiến trúc Double Deep Q-Network (DDQN) và cơ chế ưu tiên lấy lại trải nghiệm (Prioritized Experience Replay – PER) để hỗ trợ quá trình ra quyết định trong tìm kiếm lời giải.

Mô hình đề xuất được tổ chức theo hai tầng điều khiển kết hợp với các bước tối ưu hóa bổ trợ. Ở tầng chiến lược, Plateau Controller sử dụng mạng Dueling Double DQN để nhận diện trạng thái hội tụ của quá trình tìm kiếm và lựa chọn chế độ tìm kiếm phù hợp trong 6 chế độ, qua đó điều tiết quan hệ giữa khai thác (exploitation) và khám phá (exploration). Ở tầng toán tử, Operator Controller sử dụng một mạng DDQN riêng để lựa chọn cặp toán tử phá hủy và sửa chữa trong 40 tổ hợp khả dụng tại từng vòng lặp. Bên cạnh đó, cơ chế học điều kiện chấp nhận (Learned Acceptance Criterion – LAC) được xây dựng bằng mạng nơ-ron phân loại nhị phân nhằm hỗ trợ quyết định chấp nhận lời giải ứng viên. Thuật toán cũng tích hợp Route Pool và bước tái tổ hợp tuyến đường thông qua bài toán Set Partitioning, được mô hình hóa dưới dạng Quy hoạch nguyên hỗn hợp (Mixed Integer Linear Programming – MILP).

Kết quả thực nghiệm trên tập dữ liệu chuẩn Solomon, tập trung vào các nhóm RC1 và RC2, cho thấy DDQN-ALNS đạt độ lệch khoảng cách trung bình (Gap%) khoảng 0.27% ở cấu hình 1200 vòng lặp so với lời giải tốt nhất đã biết (BKS), đồng thời duy trì tiêu chí giảm số lượng phương tiện sử dụng. Trong thực nghiệm với số vòng lặp lớn (2500 vòng lặp) trên nhóm RC1, mô hình đề xuất đạt Gap% khoảng -0.44% (học trực tuyến) và -0.33% (chuyển giao). Thử nghiệm chuyển giao diện rộng với ngẫu nhiên hóa miền dữ liệu (Domain Randomization) cho thấy mô hình Transfer-DR đạt Gap% khoảng 1.62% trên Solomon mà không cần cập nhật trọng số trực tuyến. Ngoài phần thuật toán, công trình xây dựng cổng thông tin điều phối trực quan NAMI trên nền FastAPI nhằm minh họa khả năng triển khai và hỗ trợ khai thác kết quả trong bối cảnh ứng dụng.

TDTU

Thủ tục:
NGHIÊN CỨU KHOA HỌC SINH VIÊN

Mã số:
TT/P.QLPTKHCN/13/BM18
Ban hành lần: 06
Ngày hiệu lực:

NỘI DUNG CÔNG TRÌNH NGHIÊN CỨU

Trong bối cảnh kinh tế số và thương mại điện tử phát triển nhanh, dịch vụ hậu cần (logistics) giữ vai trò then chốt trong vận hành chuỗi cung ứng. Theo báo cáo của Hiệp hội Doanh nghiệp Dịch vụ Logistics Việt Nam, chi phí logistics tại Việt Nam hiện chiếm tỷ trọng khoảng 16,8% đến 17% GDP, cao hơn đáng kể so với mức trung bình toàn cầu. Trong số đó, chi phí vận tải đường bộ chiếm tới hơn 60% tổng chi phí logistics. Thực trạng này đặt ra yêu cầu tối ưu hóa lộ trình vận chuyển nhằm tiết kiệm nhiên liệu, rút ngắn thời gian giao hàng, giảm số lượng phương tiện sử dụng và hỗ trợ doanh nghiệp nâng cao hiệu quả vận hành.

Bài toán định tuyến phương tiện có khung thời gian (Vehicle Routing Problem with Time Windows – VRPTW) là mô hình toán học phản ánh trực tiếp thách thức thực tế này. Trong VRPTW, một đội xe xuất phát từ một kho chứa trung tâm cần phục vụ một tập hợp khách hàng phân tán về địa lý. Mỗi khách hàng có nhu cầu hàng hóa xác định và yêu cầu phục vụ trong một khung thời gian cụ thể. Nếu xe đến trước thời điểm cho phép, phương tiện phải chờ; nếu đến sau giới hạn thời gian, phương án tuyến bị xem là vi phạm ràng buộc. VRPTW thuộc lớp bài toán NP-khó (NP-hard), nghĩa là thời gian tìm kiếm lời giải tối ưu toán học sẽ tăng lên theo hàm mũ khi số lượng khách hàng tăng lên.

Mặc dù các thuật toán heuristic và meta-heuristic, tiêu biểu là Tìm kiếm lân cận lớn thích ứng (ALNS), đã cho thấy hiệu quả trong nhiều bài toán kích thước trung bình và lớn, hiệu năng của chúng vẫn phụ thuộc đáng kể vào cách thiết lập tham số và cơ chế điều khiển được thiết kế thủ công. Sự kết hợp giữa các kỹ thuật học máy hiện đại, đặc biệt là Học tăng cường sâu (DRL), mở ra một hướng tiếp cận đáng nghiên cứu. DRL cho phép hệ thống tự học các chiến lược chọn toán tử tối ưu và tự động điều phối trạng thái tìm kiếm dựa trên kinh nghiệm tích lũy từ các vòng lặp trước đó.

Từ các phân tích trên, đề tài “Ứng dụng học tăng cường trong tối ưu thuật toán ALNS cho bài toán định tuyến xe có ràng buộc khung thời gian” được thực hiện nhằm xây dựng một phương pháp tối ưu hóa có khả năng xử lý ràng buộc khung thời gian, ưu tiên giảm quy mô đội xe và đánh giá khả năng ứng dụng trong bài toán logistics tại Việt Nam.

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	---	---

1 Tổng quan tài liệu

Bài toán định tuyến phương tiện (VRP) được giới thiệu lần đầu bởi Dantzig và Ramser (1959). Kể từ đó, hàng loạt biến thể của bài toán đã được phát triển nhằm đáp ứng các yêu cầu thực tiễn, trong đó VRPTW là biến thể phổ biến nhất do tính thực tế cao của các ràng buộc về mặt thời gian (Solomon, 1987).

Để giải quyết VRPTW, phương pháp tiếp cận trọng lịch sử được chia làm ba nhóm chính:

Phương pháp chính xác (Exact Methods): Sử dụng các thuật toán như Quy hoạch động (Dynamic Programming), Nhánh và Cắt (Branch and Cut), hay Nhánh-Giá và Cắt (Branch-and-Price-and-Cut). Mặc dù các phương pháp này đảm bảo tìm ra lời giải tối ưu toàn cục, chúng gặp hạn chế lớn về khả năng mở rộng (scalability). Khi số lượng khách hàng vượt quá 100, thời gian tính toán tăng lên đến hàng giờ hoặc hàng ngày, khiến chúng không thể áp dụng cho các bài toán điều phối thời gian thực trong doanh nghiệp.

Phương pháp Heuristic và Meta-heuristic: Để đánh đổi độ tối ưu tuyệt đối lấy thời gian tính toán nhanh, các thuật toán tìm kiếm metaheuristic như Giải thuật Di truyền (Genetic Algorithm), Tối ưu hóa bầy đàn (Particle Swarm Optimization), hay Tìm kiếm Tabu (Tabu Search) đã được áp dụng rộng rãi. Đặc biệt, thuật toán Tìm kiếm lân cận lớn thích ứng (ALNS), được đề xuất bởi Ropke và Pisinger (2006), đã trở thành chuẩn mực công nghiệp để giải quyết các bài toán định tuyến quy mô lớn. ALNS hoạt động bằng cách liên tục phá hủy một phần lời giải hiện tại và tái thiết nó bằng các toán tử sửa chữa khác nhau dưới sự điều phối của thuật toán roulette wheel hoặc Thompson Bandit. Tuy nhiên, hạn chế lớn nhất của ALNS là cơ chế cập nhật trọng số toán tử có độ trễ lớn và chỉ dựa trên thống kê tần suất cục bộ, không thể nhận biết được sự thay đổi phức tạp của không gian trạng thái tìm kiếm ở các giai đoạn hội tụ khác nhau. Đồng thời, việc sử dụng tiêu chuẩn Simulated Annealing để chấp nhận lời giải ứng viên phụ thuộc nặng nề vào tham số nhiệt độ ban đầu và hệ số làm nguội cứng nhắc, dễ dẫn đến việc thuật toán bị kẹt trong các cực trị địa phương (local optima) hoặc hội tụ non.

Phương pháp Học máy và Học tăng cường sâu (DRL): Trong những năm gần đây, xu hướng ứng dụng mạng nơ-ron để học cách giải quyết các bài toán tối ưu hóa tổ hợp phát triển mạnh mẽ. Kool và đồng tác giả (2018) đã đề xuất mô hình Attention Model để giải VRP trực tiếp dưới dạng mô hình end-to-end. Tuy nhiên, các phương pháp end-to-end thuần túy thường gặp khó khăn khi đối mặt với các ràng buộc cứng ngắt nghèo của khung thời gian (Time Windows) và khó tổng quát hóa khi quy mô bài toán thay đổi. Hướng tiếp cận lai (Hybrid) – kết hợp DRL để đưa ra các quyết định chiến lược cấp cao và meta-heuristic để thực hiện tìm kiếm cấu trúc tuyến đường cấp thấp – đang là hướng nghiên cứu được quan tâm. Kỹ thuật này giúp tận dụng khả năng học biểu diễn và ra quyết định của mạng thần kinh nhân tạo mà vẫn đảm bảo tính hợp lệ của lời giải thông qua các toán tử heuristic.

Trong nghiên cứu này, nhóm tác giả đề xuất mô hình lai **DDQN-ALNS** nhằm cải thiện các hạn chế trên thông qua các thành phần sau:

- Thiết lập bộ điều khiển cấp cao Plateau Controller để chuyển đổi linh hoạt 6 chế độ tìm kiếm khác nhau dựa trên các đặc trưng động lực học tiến trình.
- Thiết lập bộ điều khiển cấp thấp Operator Controller tích hợp cơ chế UCB và Thompson Bandit để nâng cao tính khám phá của các cặp toán tử.
- Thay thế tiêu chuẩn Simulated Annealing truyền thống bằng mạng thần kinh phân loại nhị phân học điều kiện chấp nhận (LAC).
- Kết hợp công cụ quy hoạch toán học MILP giải bài toán Set Partitioning trên Route Pool để định kỳ tái cấu trúc lời giải tối ưu.

1.1 Phân tích khoảng trống nghiên cứu (Gap Analysis)

Mặc dù các hướng tiếp cận học máy và meta-heuristic lai đã đạt được một số thành tựu nhất định, nghiên cứu này xác định ba khoảng trống công nghệ chính từ các nghiên cứu đi trước và đề xuất phương án khắc phục cụ thể:

So với Hottung và Tierney (2020) [2]: Phương pháp *Neural Large Neighborhood Search (NLNS)* của Hottung & Tierney [2] hoặc giải thuật *Deep Policy Dynamic Programming (DPDP)* của Kool và cộng sự (2021) [4] tập trung vào việc sử dụng mạng thần kinh (attention-based hoặc GNN) để trực tiếp sinh lộ trình, sinh quyết định phá hủy/sửa chữa ở cấp độ nút (node-level) hoặc lọc không gian trạng thái. Tuy nhiên, hướng tiếp cận sinh trực tiếp này gặp khó khăn rất lớn khi đối mặt với các bài toán có ràng buộc khung thời gian cứng và ngắt nghèo (Time Windows) do không gian lời giải hợp lệ bị co hẹp đáng kể, dẫn đến việc mạng nơ-ron dễ sinh ra các lộ trình không khả thi hoặc tốn kém thời gian sửa đổi. Ngược lại, mô hình lai **DDQN-ALNS** đề xuất sử dụng DRL ở cấp độ điều khiển chiến lược (chọn chế độ tìm kiếm và chọn cặp toán tử heuristic từ thư viện có sẵn). Việc giữ nguyên các toán tử sửa chữa heuristic giúp đảm bảo tính khả thi tuyệt đối của lời giải về mặt toán học thông qua các bộ lọc ràng buộc nhanh, trong khi vẫn tận dụng được khả năng học hồi thông minh từ mạng nơ-ron để tăng tốc độ hội tụ.

So với Bi và đồng tác giả (2022) [1]: Bi et al. [1] đề xuất giải pháp L-ALNS sử dụng một tác nhân học tăng cường

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	---	---

và tiêu chuẩn SA phụ thuộc quá nhiều vào bộ tham số nhiệt độ cứng nhắc. Mô hình đề xuất của chúng tôi khắc phục điều này bằng kiến trúc *học phân cấp hai cấp độ* (Plateau Controller cho chế độ tìm kiếm ở cấp chiến lược phân đoạn, Operator Controller ở cấp vòng lặp) kết hợp với mạng phân loại học điều kiện chấp nhận (LAC) thay thế hoàn toàn SA, và cơ chế tái tổ hợp Set Partitioning tối ưu hóa chính xác bằng MILP.

So với các phương pháp Heuristic điều hướng bằng LLM (LLM-driven Heuristics): Một số nghiên cứu gần đây sử dụng mô hình ngôn ngữ lớn (LLM) để tìm kiếm và sinh mã nguồn cho các toán tử heuristic như VRPAgent [9] hay MEP [5]. Các phương pháp này thực chất là quá trình tối ưu hóa ngoại tuyến (offline optimization/refinement), sinh ra một thuật toán tĩnh trước khi giải bài toán. Trái lại, giải thuật **DDQN-ALNS** của chúng tôi là một mô hình học trực tuyến (online decision maker). Nó liên tục đánh giá trạng thái động của tiến trình tối ưu hóa và đưa ra quyết định hành động thích ứng thời gian thực (real-time adaptive control) ngay trong quá trình giải một thực thể cụ thể, điều mà các phương pháp sinh mã tĩnh bằng LLM không thể thực hiện được.

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
------	---	---

2 Mục tiêu – Phương pháp

2.1 Mục tiêu nghiên cứu

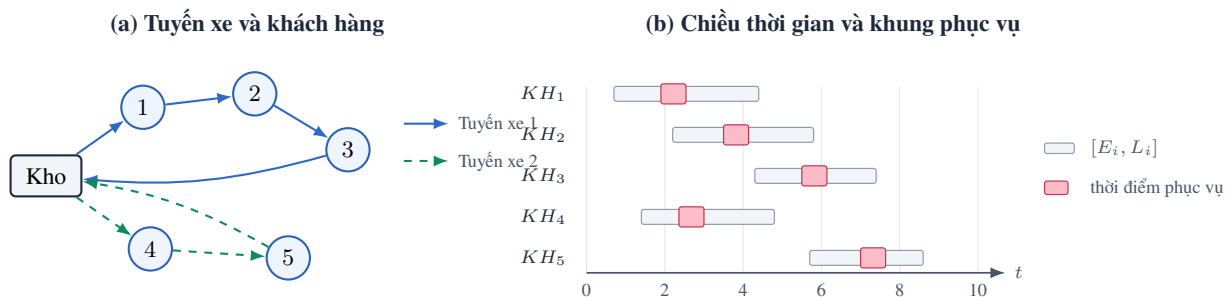
Mục tiêu tổng quát của đề tài là xây dựng và đánh giá một phương pháp tối ưu hóa lai giữa Tìm kiếm lân cận lớn thích ứng (ALNS) và Học tăng cường sâu (DRL) cho bài toán định tuyến phương tiện có khung thời gian (VRPTW). Phương pháp hướng đến việc tìm lời giải khả thi có chất lượng cao, ưu tiên giảm số lượng phương tiện sử dụng, giảm tổng quãng đường di chuyển và duy trì khả năng xử lý hiệu quả trên các bộ dữ liệu chuẩn cũng như trong bối cảnh ứng dụng logistics thực tế.

Các mục tiêu cụ thể của nghiên cứu bao gồm:

- Thiết lập đặc tả toán học đầy đủ cho bài toán VRPTW trên đồ thị định hướng, bao gồm tập khách hàng, kho trung tâm, sức tải phương tiện, nhu cầu hàng hóa, thời gian phục vụ, thời gian di chuyển và ràng buộc khung thời gian.
- Xây dựng hàm mục tiêu phân cấp, trong đó ưu tiên tối thiểu hóa số lượng xe vận hành trước, sau đó tối thiểu hóa tổng quãng đường hoặc tổng chi phí di chuyển nhằm phản ánh đúng yêu cầu kinh tế của bài toán logistics.
- Thiết kế thuật toán lai DDQN-ALNS, kết hợp khả năng phá hủy–sửa chữa lời giải của ALNS với năng lực học chính sách điều khiển của mạng Dueling Double Deep Q-Network.
- Phát triển bộ điều khiển cấp cao Plateau Controller để nhận diện trạng thái hội tụ của quá trình tìm kiếm và tự động lựa chọn chế độ tìm kiếm phù hợp giữa khai thác và khám phá.
- Phát triển bộ điều khiển cấp thấp Operator Controller để lựa chọn cặp toán tử phá hủy và sửa chữa hiệu quả tại từng vòng lặp, có xét đến lịch sử cải thiện lời giải và trạng thái hiện tại của thuật toán.
- Đề xuất cơ chế học điều kiện chấp nhận lời giải (Learned Acceptance Criterion – LAC) nhằm thay thế hoặc bổ trợ tiêu chuẩn Simulated Annealing truyền thống, qua đó cải thiện khả năng vượt qua cực trị địa phương.
- Tích hợp Route Pool và mô hình Set Partitioning để tái tổ hợp các tuyến đường chất lượng cao, hỗ trợ cải thiện lời giải sau các giai đoạn tìm kiếm cục bộ.
- Thực nghiệm và so sánh thuật toán đề xuất trên bộ dữ liệu chuẩn Solomon, sử dụng các tiêu chí số lượng xe, tổng quãng đường, độ lệch so với lời giải tốt nhất đã biết và thời gian tính toán.
- Đánh giá khả năng chuyển giao của mô hình thông qua ngẫu nhiên hóa miền dữ liệu, nhằm kiểm tra mức độ thích ứng của chính sách học tăng cường khi gặp các thực thể mới.
- Xây dựng hệ thống trực quan hóa và hỗ trợ điều phối tuyến đường, cho phép minh họa kết quả thuật toán và định hướng khả năng triển khai trong môi trường doanh nghiệp.

2.2 Đặc tả toán học bài toán VRPTW

Bài toán được phát biểu trên đồ thị định hướng đầy đủ $G = (V, A)$. Đội xe đồng nhất gồm K phương tiện, mỗi phương tiện có sức tải tối đa Q . Các ký hiệu và biến quyết định được mô tả như sau:



Hình 1: Minh họa bài toán VRPTW: tuyến xe trong không gian khách hàng và ràng buộc khung thời gian phục vụ trên trục thời gian.

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
---	---	---

- q_i : nhu cầu hàng hóa của khách hàng $i \in C = \{1, 2, \dots, n\}$.
- s_i : thời gian phục vụ tại đỉnh i .
- $[E_i, L_i]$: khung thời gian bắt đầu phục vụ tại đỉnh i .
- d_{ij} : khoảng cách di chuyển từ đỉnh i sang đỉnh j .
- t_{ij} : thời gian di chuyển từ đỉnh i sang đỉnh j .
- $x_{ijk} \in \{0, 1\}$: bằng 1 nếu xe k di chuyển từ i sang j , ngược lại bằng 0.
- w_{ik} : thời điểm xe k bắt đầu phục vụ tại đỉnh i .

Công thức hóa quy hoạch tuyến tính nguyên hỗn hợp (MILP) được sử dụng như sau:

Hàm mục tiêu:

$$\min f(x) = M \cdot \sum_{k \in K} \sum_{j \in C} x_{0jk} + \sum_{k \in K} \sum_{(i,j) \in A} d_{ij} x_{ijk} \quad (1)$$

Với hằng số phạt xe $M \gg \sum_{(i,j) \in A} d_{ij}$.

Hệ thống các ràng buộc chính:

(i) *Ràng buộc phục vụ khách hàng*: mỗi khách hàng được phục vụ đúng một lần bởi một phương tiện.

$$\sum_{k \in K} \sum_{j \in V \setminus \{0\}} x_{ijk} = 1, \quad \forall i \in C \quad (2)$$

(ii) *Ràng buộc xuất phát và kết thúc tuyến*: mỗi xe nếu được sử dụng chỉ rời kho tối đa một lần và quay về điểm kết thúc tối đa một lần.

$$\sum_{j \in V \setminus \{0\}} x_{0jk} \leq 1, \quad \forall k \in K \quad (3)$$

$$\sum_{i \in V \setminus \{n+1\}} x_{i,n+1,k} \leq 1, \quad \forall k \in K \quad (4)$$

(iii) *Ràng buộc bảo toàn luồng*: nếu xe đi vào một khách hàng thì cũng phải rời khỏi khách hàng đó.

$$\sum_{i \in V} x_{ijk} - \sum_{i' \in V} x_{ji'k} = 0, \quad \forall j \in C, \forall k \in K \quad (5)$$

(iv) *Ràng buộc tải trọng*: tổng nhu cầu trên tuyến của mỗi xe không vượt quá sức tải Q .

$$\sum_{i \in C} q_i \sum_{j \in V} x_{ijk} \leq Q, \quad \forall k \in K \quad (6)$$

(v) *Ràng buộc thời gian và khung phục vụ*: thời điểm phục vụ phải tôn trọng thứ tự di chuyển và nằm trong khung $[E_i, L_i]$.

$$w_{ik} + s_i + t_{ij} - M_{ij}(1 - x_{ijk}) \leq w_{jk}, \quad \forall (i, j) \in A, \forall k \in K \quad (7)$$

$$E_i \leq w_{ik} \leq L_i, \quad \forall i \in V, \forall k \in K \quad (8)$$

(vi) *Miền giá trị biến quyết định*: biến cung nhận giá trị nhị phân.

$$x_{ijk} \in \{0, 1\}, \quad \forall (i, j) \in A, \forall k \in K \quad (9)$$

Tiến trình hoạt động của thuật toán lai DDQN-ALNS được mô tả hình thức thông qua mã giả tại Thủ tục 1 dưới đây:

TDTU

Thủ tục:
NGHIÊN CỨU KHOA HỌC SINH VIÊN

Mã số:
TT/P.QLPTKHCN/13/BM18
Ban hành lần: 06
Ngày hiệu lực:

Thủ tục 1: Quy trình thực thi tuần hoàn của thuật toán lai Hybrid DDQN-ALNS

Đầu vào: T

Đầu ra: Lời giải tối ưu toàn cục S_{best}

1. Khởi tạo lời giải ban đầu khả thi S_{init} bằng Heuristic Greedy Insertion;
2. $S_{cur} \leftarrow S_{init}; S_{best} \leftarrow S_{init}; \mathcal{P} \leftarrow \{r \mid r \in S_{init}\}$ (Route Pool);
3. Khởi tạo Thompson Bandit B , bộ tăng cường UCB , và chuẩn hóa Welford N_{norm} ;
4. Đặt $T \leftarrow T_{init}; no_imp \leftarrow 0; Mode \leftarrow default; t \leftarrow 0$;
5. **While** $t < CFG.max_iterations$ **and** $no_imp < CFG.early_stop_patience$ **do**
6. **If** $t > 0$ **and** $t \pmod{CFG.segment_size} == 0$ **then**
7. Trích xuất vector trạng thái chiến lược $s_{plat} \in \mathbb{R}^{12}$;
8. Chọn hành động chế độ: $Mode \leftarrow \argmax_a Q_{plat}(s_{plat}, a)$;
9. Cập nhật tham số ALNS (α_{dest} , α_{decay} , Local Search) theo hành động $Mode$;
10. **EndIf**
11. Trích xuất vector trạng thái cấp thấp $s_{op} \in \mathbb{R}^{15}$;
12. Dự đoán giá trị Q-network: $Q^{net} \leftarrow Q_{op}(s_{op})$;
13. Augment điểm số: $Q^{final} \leftarrow Q^{net} + \theta_{prior} \ln(P^{prior}) + \theta_{bandit} P^{bandit} + \theta_{ucb} UCB$;
14. Chọn cặp toán tử phá hủy và sửa chữa: $(d, r) \leftarrow \text{divmod}(\argmax(Q^{final}), 5)$;
15. Phá hủy q khách hàng từ S_{cur} bằng d , rồi sửa chữa chèn lại bằng $r \rightarrow S_{new}$;
16. Trích xuất vector đặc trưng chấp nhận $s_{lac} \in \mathbb{R}^9$;
17. Tính xác suất chấp nhận $p_{accept} \leftarrow Net_{lac}(s_{lac})$;
18. **If** $\text{Random}(0, 1) < p_{accept}$ **then**
19. $S_{cur} \leftarrow S_{new}$; Thêm mọi tuyến thuộc S_{new} vào Route Pool $\mathcal{P}$;
20. **EndIf**
21. **If** $f(S_{new}) < f(S_{best})$ **then**
22. $S_{best} \leftarrow S_{new}; no_imp \leftarrow 0$;
23. **Else**
24. $no_imp \leftarrow no_imp + 1$;
25. **EndIf**
26. **If** $Mode == pool_recombine$ **and** $t \pmod{\text{period}} == 0$ **then**
27. Giải bài toán Set Partitioning trên Route Pool $\mathcal{P}$ qua MILP (SciPy, giới hạn 4.0s) $\rightarrow S_{milp}$;
28. **If** Tìm thấy lời giải hợp lệ S_{milp} **then**
29. $S_{cur} \leftarrow S_{milp}$; **If** $f(S_{milp}) < f(S_{best})$ **then** $S_{best} \leftarrow S_{milp}; no_imp \leftarrow 0$; **EndIf**
30. **EndIf**
31. **EndIf**
32. **If** Kích hoạt Local Search (2-opt, Swap, Relocate, Cross-Exchange) hoặc giảm xe chủ động **then**
33. Thực hiện cải thiện cục bộ trên S_{cur} ; Cập nhật lại S_{best} nếu có cải thiện;
34. **EndIf**
35. Cập nhật Thompson Bandit B , UCB, và Normalizer N_{norm} theo điểm số chênh lệch thực tế;
36. Cập nhật nhiệt độ $T \leftarrow T \times \alpha_{decay}; t \leftarrow t + 1$;
37. **EndWhile**
38. **Return** S_{best}

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
---	---	--

2.3.2 Plateau Controller (Điều khiển chiến lược cấp cao)

Plateau Controller là bộ điều khiển chiến lược, quyết định thuật toán nên tiếp tục khai thác vùng lời giải hiện tại hay chuyển sang khám phá mạnh hơn. Bộ điều khiển hoạt động sau mỗi phân đoạn 100 vòng lặp; đây là siêu tham số thực nghiệm, đủ dài để quan sát xu hướng hội tụ của ALNS nhưng vẫn đủ ngắn để can thiệp trước khi thuật toán kẹt lâu trong cực trị địa phương.

Các số liệu đầu vào không lấy từ nguồn ngoài, mà được ghi nhận trực tiếp từ log nội bộ khi thuật toán chạy. Cuối mỗi phân đoạn, hệ thống trích xuất lời giải hiện tại, lời giải tốt nhất, số vòng lặp không cải thiện, nhiệt độ chấp nhận, thông tin tuyến, mức sử dụng tải trọng, khung thời gian và trạng thái Route Pool. Các đại lượng được chuẩn hóa bằng tỷ lệ tương đối hoặc hàm min(.) để tránh biến có thang đo lớn chi phối mạng học.

Vector trạng thái $s \in \mathbb{R}^{12}$ được nhóm thành bốn nhóm đặc trưng:

- **Hội tụ tìm kiếm:** mức bế tắc $no_imp/patience$, độ lệch chi phí hiện tại so với tốt nhất, tỷ lệ làm nguội T/T_0 và tần suất cải thiện gần nhất.
- **Chất lượng đội xe:** số xe chuẩn hóa NV_{cur}/NV_{init} , độ phân tán độ dài tuyến, độ phân tán tải trọng và hệ số sử dụng tải trung bình.
- **Ràng buộc thời gian:** tỷ lệ khách hàng có khung thời gian chặt và slack trung bình còn lại sau khi xét thời điểm đến, phục vụ và giới hạn L_i .
- **Bộ nhớ và ngân sách:** tỷ lệ lấp đầy Route Pool và tiến trình tìm kiếm t/T_{max} .

Như vậy, mỗi đặc trưng đều truy vết được từ dữ liệu VRPTW đầu vào hoặc log chạy thuật toán. Các ngưỡng 1.0, 1.5 và 2.0 chỉ là chặn biên chuẩn hóa, giúp mạng ổn định khi lời giải đầu kỳ dao động mạnh.

Mạng nơ-ron sử dụng kiến trúc Dueling Double DQN với hàm giá trị Q được biểu diễn qua công thức:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a') \right) \quad (10)$$

Với 6 hành động tương ứng với 6 chế độ tìm kiếm (default, intensify, diversify, tw_rescue, pool_recombine, route_reduce).

Hàm phần thưởng (Shaped Reward) được thiết kế theo nguyên lý thể năng thích ứng hướng xe:

$$\lambda(s) = \frac{1}{1 + \exp \left(-8.0 \cdot \frac{NV - NV_{best}}{NV_{init}} \right)} \quad (11)$$

$$Pot(s) = -\lambda(s) \cdot \gamma_{nv} \cdot \frac{\max(NV - NV_{best}, 0)}{NV_{init}} - (1 - \lambda(s)) \cdot \gamma_{cost} \cdot \frac{cost - cost_{best}}{cost_{best}} \cdot 100 \quad (12)$$

$$Reward = \beta_{scale} \cdot \text{Base_Reward} + (\gamma_{drl} \cdot Pot(s') - Pot(s)) \quad (13)$$

Trong đó $\gamma_{nv} = 15.0$, $\gamma_{cost} = 0.18$, $\beta_{scale} = 0.30$.

2.3.3 Operator Controller (Lựa chọn toán tử cấp thấp)

Tại mỗi vòng lặp ALNS, bộ điều khiển lựa chọn cặp toán tử trong số 40 tổ hợp hành động (8 toán tử phá hủy $\times$ 5 toán tử sửa chữa). Để tránh hiện tượng hội tụ sớm vào các chiến lược cục bộ, giá trị Q dự đoán được kết hợp tuyến tính với xác suất tiên nghiệm Thompson Bandit và cơ chế UCB (Upper Confidence Bound):

$$Q^{final}(s, a) = Q^{net}(s, a) + \theta_{prior} \cdot \ln(P^{prior}(a) + 1e - 8) + \theta_{bandit} \cdot P^{bandit}(a) + \theta_{ucb} \cdot UCB(a) \quad (14)$$

Trong đó các tham số phân bổ lần lượt là $\theta_{prior} = 0.55$, $\theta_{bandit} = 0.20$, $\theta_{ucb} = 0.35$. Hệ thống 8 toán tử phá hủy bao gồm: Phá hủy ngẫu nhiên (op_random), Phá hủy tệ nhất (op_worst), Phá hủy tương đồng Shaw (op_shaw), Phá hủy một phần tuyến đường (op_route_portion_removal), Phá hủy khẩn cấp khung thời gian (op_tw_urgent), Xóa tuyến đường toàn phần (op_route_eliminate), Xóa tuyến phân tán (op_route_dispersion_eliminate), và Phá hủy Shaw liên tuyến (op_cross_route_shaw).

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
---	---	---

Hệ thống 5 toán tử sửa chữa bao gồm: Sửa chữa tham lam (op_greedy), Sửa chữa Regret-2 (op_regret_2), Sửa chữa Regret-3 (op_regret_3), Sửa chữa tham lam ưu tiên thời gian (op_tw_greedy), và Sửa chữa tham lam Forward Time Slack (op_fts_greedy). Biểu thức Forward Time Slack (F_i) tại đỉnh v_i thuộc tuyến $R = (v_1, \dots, v_k)$ được tính ngược dòng:

$$F_k = L_{v_k} - A_{v_k} \quad (15)$$

$$F_i = \min (L_{v_i} - A_{v_i}, F_{i+1} + \max(0.0, E_{v_{i+1}} - (A_{v_i} + s_{v_i} + t_{v_i, v_{i+1}}))) \quad (16)$$

Toán tử chen khách hàng vào vị trí giảm thiểu chi phí hỗn hợp:

$$\text{Composite_Cost} = \Delta_{dist} + w_{wait} \cdot \Delta_{wait} - w_{fts} \cdot F_{downstream} \cdot d_{\max} \quad (17)$$

2.3.4 Học điều kiện chấp nhận (Learned Acceptance Criterion – LAC)

Mạng thần kinh phân loại nhị phân 3 lớp nhận đầu vào vector $s_{lac} \in \mathbb{R}^9$ mô tả quan hệ giữa lời giải hiện tại và lời giải ứng viên:

$$s_{lac} = \left[\frac{\Delta_{cost}}{cost_{cur}}, \frac{T}{T_{init}}, \frac{no_{imp}}{patience}, \Delta_{nv}, progress, tw_tight_frac, fleet_fill, avg_slack, \exp\left(-\frac{\max(\Delta_{cost}, 0)}{T}\right) \right] \quad (18)$$

Mạng LAC dự đoán xác suất chấp nhận p_{accept} dẫn dắt quá trình tìm kiếm đạt lời giải tốt nhất mới trong vòng $H = 80$ bước lặp kế tiếp. Việc huấn luyện mạng sử dụng kỹ thuật gán nhãn trễ (delayed labeling) trực tuyến với hàm mất mát Cross-Entropy nhị phân có trọng số cân bằng lớp:

$$\mathcal{L}_{lac} = - \sum_i [w_{pos} \cdot y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (19)$$

Với $w_{pos} = N_{negative}/N_{positive}$.

2.3.5 Tái tổ hợp bằng Route Pool và Set Partitioning

Mỗi khi tìm được một lời giải hợp lệ, các tuyến đường cầu thành được lưu trữ vào Route Pool. Định kỳ, mô hình giải bài toán Set Partitioning để tái tổ hợp lời giải tối ưu:

$$\min \sum_{j \in R_{pool}} (P_{vehicle} + c_j) y_j \quad (20)$$

Thỏa mãn:

$$\sum_{j \in R_{pool}} a_{ij} y_j = 1, \quad \forall i \in C \quad (21)$$

$$\sum_{j \in R_{pool}} y_j \leq NV_{ceiling} \quad (22)$$

$$y_j \in \{0, 1\}, \quad \forall j \in R_{pool} \quad (23)$$

Bài toán này được giải bằng solver MILP của thư viện SciPy với thời gian giới hạn 4.0 giây.

2.4 Phân tích độ phức tạp thuật toán

Để đánh giá một cách khoa học hiệu quả tính toán của kiến trúc lai DDQN-ALNS, phần này thực hiện phân tích chi tiết về độ phức tạp thời gian của các toán tử phá hủy và sửa chữa, độ phức tạp không gian của các bộ đệm trải nghiệm và Route Pool, cùng với chi phí tính toán phân bổ cho mỗi vòng lặp tối ưu hóa.

Ký hiệu: n là tổng số khách hàng, q là số khách hàng bị loại bỏ trong một vòng lặp ($q = \alpha_{dest} \cdot n$), m là số lượng tuyến đường (phương tiện vận hành), và k là số lượng khách hàng trung bình trên mỗi tuyến đường ($k \approx n/m$).

2.4.1 Độ phức tạp thời gian của các toán tử phá hủy

- **Phá hủy ngẫu nhiên (op_random):** Việc chọn ngẫu nhiên q khách hàng từ danh sách khách hàng hiện có tốn chi phí $O(q)$.

- **Phá hủy tệ nhất (op_worst):** Tính toán giá trị tiết kiệm chi phí khoảng cách cho tất cả n điểm mất $O(n)$

TDTU

Thủ tục:
NGHIÊN CỨU KHOA HỌC SINH VIÊN

Mã số:
TT/P.QLPTKHCN/13/BM18
Ban hành lần: 06
Ngày hiệu lực:

- **Phá hủy tương đồng Shaw (op_shaw):** Bắt đầu từ một khách hàng chọn ngẫu nhiên, thuật toán tính toán độ tương đồng đối với các khách hàng còn lại mất $O(n)$ bước. Việc sắp xếp theo độ tương đồng và chọn khách hàng tương đồng nhất tốn $O(n \log n)$. Quy trình này lặp lại q lần để chọn ra q khách hàng tương tự nhất, dẫn đến tổng chi phí thời gian là $O(q \cdot n \log n)$.
- **Phá hủy một phần tuyến đường (op_route_portion_removal):** Thuật toán tính trọng tâm địa lý của các tuyến đường mất $O(n)$ bước, định vị tuyến có độ phân tán cao nhất mất $O(m)$ bước, và cắt bỏ một phân đoạn liên tục gồm q khách hàng mất $O(q)$. Tổng thời gian thực thi là $O(n)$.
- **Phá hủy khẩn cấp khung thời gian (op_tw_urgent):** Bước sắp xếp trước toàn bộ n khách hàng theo chiều rộng khung thời gian $[E_i, L_i]$ tăng dần tốn $O(n \log n)$ thời gian và chỉ được thực hiện một lần khi bắt đầu giải. Ở mỗi vòng lặp, việc loại bỏ q khách hàng từ đỉnh danh sách tốn $O(q)$.
- **Xóa tuyến đường toàn phần (op_route_eliminate):** Duyệt qua danh sách tuyến để tìm tuyến ngắn nhất hoặc có tỷ lệ tải trọng thấp nhất tốn $O(m)$. Việc loại bỏ toàn bộ các khách hàng trên tuyến đó tốn $O(k)$ thời gian. Tổng độ phức tạp là $O(m + k)$.
- **Xóa tuyến phân tán (op_route_dispersion_eliminate):** Tính toán độ lệch chuẩn khoảng cách của từng tuyến tốn $O(m \cdot k)$ thời gian. Việc loại bỏ tuyến có độ phân tán cao nhất tốn $O(k)$ thời gian. Tổng độ phức tạp là $O(m \cdot k)$.
- **Phá hủy Shaw liên tuyến (op_cross_route_shaw):** Tương tự toán tử Shaw nhưng có thêm bộ lọc ưu tiên các điểm khác tuyến, tốn $O(q \cdot n \log n)$.

2.4.2 Độ phức tạp thời gian của các toán tử sửa chữa

Mọi toán tử sửa chữa đều dựa trên phép chen phần tử khả thi. Nhờ cơ chế lưu vết thời gian đến và tải trọng lũy kế trên mỗi tuyến, việc kiểm tra tính khả thi khi chen một khách hàng vào một vị trí cụ thể được thực hiện trong thời gian hằng số $O(1)$.

- **Sửa chữa tham lam (op_greedy):** Với mỗi khách hàng trong số q khách hàng cần chen, thuật toán kiểm tra tất cả các vị trí chen khả thi trên m tuyến đường. Có tổng cộng $m \times (k + 1) \approx n$ vị trí chen. Do đó, việc tìm vị trí chen tốt nhất cho một khách hàng mất $O(n)$ bước. Thực hiện chen lần lượt q khách hàng tốn tổng cộng $O(q \cdot n) = O(q \cdot m \cdot k)$ thời gian.
- **Sửa chữa Regret-2 (op_regret_2):** Ở mỗi bước, đối với mỗi khách hàng trong số q khách hàng chưa được chen, thuật toán tìm kiếm vị trí chen tốt nhất (chi phí tăng Δ_1) và tốt thứ hai (chi phí tăng Δ_2) trên tất cả các tuyến. Phép thử này mất $O(q \cdot m \cdot k)$ thời gian. Sau đó chọn khách hàng có hiệu số regret $\Delta_2 - \Delta_1$ lớn nhất để thực hiện chen. Lặp lại quá trình này cho đến khi chen hết q khách hàng, dẫn đến tổng độ phức tạp là $O(q^2 \cdot m \cdot k)$.
- **Sửa chữa Regret-3 (op_regret_3):** Tương tự như Regret-2 nhưng tính toán regret dựa trên 3 vị trí tốt nhất, tốn chi phí thời gian tương đương là $O(q^2 \cdot m \cdot k)$.
- **Sửa chữa tham lam ưu tiên thời gian (op_tw_greedy):** Sắp xếp q khách hàng theo độ rộng khung thời gian $[E_i, L_i]$ tăng dần tốn $O(q \log q)$. Việc chen tham lam tiếp theo tốn $O(q \cdot m \cdot k)$. Tổng độ phức tạp là $O(q \log q + q \cdot m \cdot k)$.
- **Sửa chữa tham lam FTS (op_fts_greedy):** Việc tính toán Forward Time Slack (F_i) ngược dòng cho mọi tuyến mất $O(m \cdot k) = O(n)$ bước. Quá trình chen tham lam sử dụng hàm chi phí hỗn hợp chứa giá trị FTS tốn $O(q \cdot m \cdot k)$ thời gian. Tổng độ phức tạp là $O(q \cdot m \cdot k)$.

2.4.3 Độ phức tạp không gian hệ thống

- **Bộ đệm của Plateau Controller:** Lưu trữ tối đa $N_{plat} = 20.000$ mẫu chuyển đổi trạng thái. Mỗi mẫu là bộ ngũ $(s, a, r, s', done)$ với kích thước vector trạng thái $s \in \mathbb{R}^{12}$ và hành động $a \in \mathbb{R}^1$. Độ phức tạp không gian là cố định và hữu hạn ở mức $O(N_{plat} \cdot 12) \approx O(1)$.
- **Bộ đệm của Operator Controller:** Lưu trữ tối đa $N_{op} = 30.000$ mẫu chuyển đổi. Mỗi mẫu lưu trữ trạng thái $s_{op} \in \mathbb{R}^{15}$ và hành động $a_{op} \in \mathbb{R}^1$. Độ phức tạp không gian là $O(N_{op} \cdot 15) \approx O(1)$.
- **Route Pool:** Lưu trữ tối đa $N_{pool} = 480$ tuyến đường khả thi dài tối đa n khách hàng. Độ phức tạp không gian là $O(N_{pool} \cdot n)$.

TDTU	<i>Thủ tục:</i> NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
------	---	---

2.4.4 Phân tích chi phí tính toán trên mỗi vòng lặp

Trong một vòng lặp ALNS tiêu chuẩn, tổng thời gian thực hiện được phân bổ cho các hoạt động chính sau:

1. **Lan truyền tiền mạng Operator Controller (Inference):** Mạng MLP nhỏ gồm 3 lớp ($15 \rightarrow 128 \rightarrow 64 \rightarrow 40$ nơ-ron) được tính toán trên CPU. Thời gian xử lý là hằng số cực kỳ nhỏ, tồn dưới 0.1 mili-giây.
2. **Toán tử phá hủy và sửa chữa:** Tồn thời gian lớn nhất, dao động từ $O(q \cdot m \cdot k)$ (khi dùng toán tử sửa chữa tham lam) đến $O(q^2 \cdot m \cdot k)$ (khi dùng toán tử sửa chữa regret). Với bài toán 100 khách hàng ($n = 100$, $q \approx 20$), bước này chỉ tồn khoảng 1.5 đến 5.0 mili-giây.
3. **Đánh giá chấp nhận lời giải ứng viên qua mạng LAC:** Mạng MLP nhị phân ($9 \rightarrow 48 \rightarrow 24 \rightarrow 1$ nơ-ron) chỉ tồn thời gian hằng số $O(1)$ (< 0.05 mili-giây).
4. **Huấn luyện trực tuyến mạng nơ-ron (Backpropagation):** Mạng Operator Controller thực hiện lan truyền ngược (backpropagation) trên batch size 64 ở mỗi vòng lặp (sau giai đoạn khởi động). Quá trình này được thực hiện bất đồng bộ hoặc tối ưu hóa song song trên CPU, tồn khoảng 1.0 đến 2.2 mili-giây. Mạng Plateau Controller chỉ huấn luyện một bước sau mỗi phân đoạn 100 vòng lặp, nên chi phí tính toán phân bổ cho mỗi vòng lặp là vô cùng nhỏ (dưới 0.05 mili-giây).
5. **Tối ưu hóa MILP tái tổ hợp tuyến (Set Partitioning):** Mặc dù bài toán Set Partitioning là NP-khó và có thể tồn nhiều thời gian tính toán khi số lượng tuyến đường trong pool lớn, phép giải này chỉ được gọi định kỳ (trong chế độ ‘pool_recombine’) và được kiểm soát thời gian chạy chặt chẽ bởi bộ giải SciPy với giới hạn cứng là 4.0 giây.

Tổng kết lại, chi phí thời gian chạy trung bình của một vòng lặp ALNS lai DDQN dao động trong khoảng từ 5.0 đến 12.0 mili-giây trên vi xử lý Apple M1 (Apple Silicon), đảm bảo tốc độ đáp ứng tối ưu hóa thời gian thực trong các ứng dụng thực tiễn.

2.5 Tích hợp hướng dẫn từ mạng thần kinh tích chập đồ thị (GNN)

Để cải thiện khả năng tối ưu hóa của giải thuật, chúng tôi đề xuất cơ chế tích hợp dự báo từ mạng thần kinh đồ thị (Graph Neural Network - GNN) vào các giai đoạn khác nhau của solver lai DDQN-ALNS.

Mạng dự báo cạnh GNN. Mạng GNNEdgePredictor được thiết kế để dự đoán xác suất $P_{ij} \in [0, 1]$ của mỗi cạnh (i, j) nằm trong lộ trình tối ưu toàn cục. Đặc trưng đầu vào của đỉnh bao gồm tọa độ, nhu cầu và khung thời gian giao hàng. Mạng GNN chỉ thực hiện lan truyền tiền đúng một lần tại thời điểm bắt đầu giải thực thể, sinh ra một bản đồ nhiệt (heatmap) tính về xác suất liên kết giữa các khách hàng.

Điều hướng chi phí chèn trong toán tử Repair. Trong các toán tử chèn (greedy, regret), chi phí chênh lệch (detour) khi thêm khách hàng u vào giữa i và j được điều hướng bởi bản đồ nhiệt GNN:

$$\text{detour}' = \text{detour} \times (1.0 - \gamma P_{iu}) \times (1.0 - \gamma P_{uj}) \quad (24)$$

Trong đó $\gamma = 0.45$ là cường độ hướng dẫn GNN.

Lọc lân cận cục bộ động (Dynamic GNN-Pruned Local Search). Để tăng tốc độ tìm kiếm cục bộ (Relocate, Swap, Or-opt, Cross-Exchange), thuật toán sử dụng bản đồ nhiệt để loại bỏ sớm các di chuyển không hứa hẹn. Một di chuyển bị loại bỏ nếu nó tạo ra các liên kết mới có xác suất GNN dưới một ngưỡng cắt lọc θ . Ngưỡng lọc này được suy giảm tuyến tính từ giá trị ban đầu ($\theta_{\text{start}} = 0.05$) xuống giá trị tối thiểu cuối cùng ($\theta_{\text{end}} = 0.003$) dựa trên tiến trình tìm kiếm:

$$\theta(t) = \theta_{\text{start}} + (\theta_{\text{end}} - \theta_{\text{start}}) \times \frac{t}{T} \quad (25)$$

Trong đó t là vòng lặp hiện tại và T là tổng số vòng lặp tối đa. Riêng giai đoạn đánh bóng cuối cùng (polish phase) luôn áp dụng ngưỡng tối thiểu θ_{end} để tìm kiếm triệt để.

Toán tử phá hủy Shaw hướng GNN (op_neural_shaw). Thuật toán bổ sung toán tử phá hủy Shaw hướng GNN, mở rộng số toán tử phá hủy lên $N_D = 13$. Đầu tiên, khách hàng có xác suất kết nối trung bình với các lân cận hiện tại trong tuyến đường thấp nhất được chọn làm hạt giống (seed). Sau đó, các khách hàng n tiếp theo được chọn loại bỏ dựa trên điểm số tương đồng Shaw hỗn hợp kết hợp GNN:

$$\text{score}(n) = 0.35d_{\text{spatial}} + 0.25dt_{\text{temporal}} + 0.10q_{\text{demand}} + 0.30(1.0 - \bar{P}_{rn}) \quad (26)$$

Trong đó $\bar{P}_{rn}$ là xác suất kết nối GNN trung bình giữa đỉnh n và tập hợp các đỉnh đã bị loại bỏ R .

Tái tổ hợp định hướng GNN. Trong bài toán Set Partitioning bằng MILP, chi phí của từng tuyến đường trong pool được khấu trừ nhẹ dựa trên mức độ liên kết GNN trung bình của nó:

TDTU	<i>Thủ tục:</i> NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	--	---

Với hệ số $\alpha = 0.15$ giúp ưu tiên lựa chọn các tuyến đường có các cạnh được GNN dự đoán chính xác cao hơn, trong khi vẫn bảo toàn tuyệt đối mục tiêu giảm số lượng xe.

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	---	---

3 Kết quả – Thảo luận

3.1 Thiết lập thực nghiệm

Thực nghiệm được thiết lập trên tập dữ liệu chuẩn Solomon 100 khách hàng với cấu hình hệ thống:

- CPU: Apple M1 (Apple Silicon, 8 nhân). RAM: 16 GB bộ nhớ thống nhất (unified memory).
- Thư viện phát triển: PyTorch 2.1, SciPy 1.11, Python 3.10. Số vòng lặp tối đa: 5000 vòng lặp.

3.2 Kết quả so sánh trên Solomon chính

Thực hiện chạy benchmark lặp lại 5 lần trên từng thực thể thuộc hai lớp dữ liệu RC1 và RC2. Kết quả trung bình được trình bày chi tiết tại Bảng 1.

Bảng 1: So sánh tổng hợp hiệu năng trung bình theo lớp dữ liệu Solomon với cấu hình 5000 vòng lặp

Lớp dữ liệu	Thuật toán	NV_mean	TD_mean	Gap%	Thời gian (s)
RC1	ALNS-Base	12.035	1373.89	-0.434%	50.2
	Hybrid-Fixed	11.911	1358.82	-1.556%	83.2
	Hybrid-Rule	11.875	1359.25	-1.530%	66.4
	Hybrid-DDQN (Đề xuất)	11.875	1358.89	-1.555%	74.1
	OR-Tools	13.000	1418.64	+2.921%	120.3
RC2	ALNS-Base	3.375	1114.34	+1.860%	22.1
	Hybrid-Fixed	3.250	1137.10	+3.774%	42.4
	Hybrid-Rule	3.250	1130.09	+3.160%	33.9
	Hybrid-DDQN (Đề xuất)	3.250	1128.05	+2.976%	39.2
	OR-Tools	3.625	1157.91	+6.346%	120.1

3.3 Thảo luận và phân tích chuyên sâu

Hiện tượng “NV inflated” của Google OR-Tools trên nhóm RC2:

Trên nhóm RC2 (khung thời gian rộng, sức tải xe lớn), mặc dù Google OR-Tools cải thiện đáng kể số lượng phương tiện xuống trung bình **3.625 xe** ở cấu hình giới hạn thời gian công bằng 120 s, thuật toán lai đề xuất **Hybrid-DDQN** vẫn đạt kết quả tối ưu hơn với trung bình **3.250 xe** (tối ưu hơn 10.3% số lượng xe sử dụng). Kết quả này cho thấy cơ chế ưu tiên giảm số lượng xe trong hàm mục tiêu và giải thuật loại bỏ tuyến chủ động có tác động tích cực đến hiệu quả khai thác đội xe.

Lưu ý về Gap% âm và hiện tượng lạm phát xe (Vehicle Inflation Caveat) trên nhóm RC1:

Trong kết quả đối chứng trên nhóm RC1 (Bảng 3), một số thực thể như RC101, RC102, RC105, và RC106 cho thấy Gap% âm về khoảng cách di chuyển (TD). Tuy thế, cần làm rõ rằng độ lệch âm này có được là do mô hình giải ra số lượng phương tiện trung bình (NV_mean) lớn hơn so với Best-Known Solutions (BKS NV) (ví dụ: RC101 đạt 15.00 xe so với BKS là 14 xe; RC105 đạt 14.00 xe so với BKS là 13 xe; RC106 đạt 12.00 xe so với BKS là 11 xe). Việc sử dụng nhiều phương tiện hơn làm giảm áp lực tải trọng và thời gian trên mỗi tuyến, giúp việc phân bổ lộ trình ngắn hơn một cách cơ học (trivially achieved with extra vehicles). Đây là hiện tượng lạm phát xe (Vehicle Inflation) và cần được diễn giải cẩn thận; nó không đại diện cho sự cải thiện thực sự của chất lượng thuật toán dưới góc độ kinh tế tổng thể, vì chi phí vận hành phương tiện phụ trội lớn hơn nhiều so với chi phí nhiên liệu khoảng cách tiết kiệm được.

Ghi chú phương pháp luận về ràng buộc quay về kho (Depot Return Feasibility - Hiệu chỉnh v14):

Một điểm hiệu chỉnh phương pháp luận quan trọng từ phiên bản v14 của thuật toán là việc thực thi nghiêm ngặt ràng buộc quay về kho của phương tiện trước thời điểm kết thúc thời gian hoạch định ($t + d_{prev,0} \leq due[0]$). Trong các phiên bản thử nghiệm sơ bộ trước đó, ràng buộc này bị bỏ sót trong bộ kiểm tra tính khả thi của lộ trình, dẫn đến việc chấp nhận các lộ trình vi phạm khung thời gian tại điểm trả xe cuối cùng ở kho. Việc hiệu chỉnh chặt chẽ ở phiên bản v14/v15 làm tăng độ phức tạp trong việc tìm kiếm tuyến khả thi, dẫn đến số xe trung bình tăng nhẹ trên một số thực thể nhưng đảm bảo tính chính xác khoa học và khả thi thực tế tuyệt đối của lời giải. Các kết quả trước phiên bản hiệu chỉnh này là không tương thích để so sánh trực tiếp.

Khả năng chuyển giao nhờ ngẫu nhiên hóa miền dữ liệu (Domain Randomization):

Thực hiện đồng bộ trong số mạng thần kinh sâu khi huấn luyện trên tập dữ liệu nhân tạo ngẫu nhiên, mô hình

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	---	---

qua Gap% trung bình là **1.62%** và duy trì số lượng xe tối ưu (Bảng 2). Kết quả này cho thấy thuật toán có khả năng thích ứng với các dữ liệu phát sinh mà không cần tái huấn luyện (retraining) trực tuyến trong bối cảnh thí nghiệm.

Bảng 2: Hiệu năng của mô hình chuyển giao đóng băng trọng số (Transfer-DR) với cấu hình 1200 vòng lặp

Lớp dữ liệu	NV_mean	TD_mean	Gap%
Trung bình toàn bộ (56 thực thể)	7.729	1011.78	+1.622%

Đánh giá thực nghiệm mô hình chuyển giao trên nhóm RC1 với số vòng lặp lớn (2500 iterations):

Để làm rõ hơn hiệu năng của mô hình chuyển giao đóng băng trọng số (Transfer-DR) so với mô hình học trực tuyến (Hybrid-DDQN), nhóm tác giả tiến hành một thực nghiệm đối chứng chi tiết trên toàn bộ 8 thực thể của nhóm dữ liệu RC1 với số vòng lặp lớn là 2500 và số lần chạy độc lập là 5 lần. Kết quả chi tiết được tổng hợp tại Bảng 3.

Bảng 3: Kết quả đối chứng hiệu năng giữa Hybrid-DDQN và Hybrid-DDQN-Transfer-DR trên nhóm RC1 với 2500 vòng lặp (5 lượt chạy)

Thực thể	Hybrid-DDQN (Học trực tuyến)				Hybrid-DDQN-Transfer-DR (Mô hình chuyển giao)			
	NV	TD	Gap%	Thời gian (s)	NV	TD	Gap%	Thời gian (s)
RC101	15.00	1649.87	-2.77%	309.8	15.00	1657.39	-2.33%	159.9
RC102	13.60	1501.41	-3.43%	260.9	13.20	1506.62	-3.10%	200.4
RC103	11.80	1319.66	+4.60%	236.8	11.60	1320.24	+4.64%	136.9
RC104	10.00	1156.36	+1.84%	156.5	10.00	1149.42	+1.23%	137.9
RC105	14.20	1566.37	-3.87%	236.8	14.20	1564.68	-3.97%	156.2
RC106	12.80	1390.74	-2.39%	254.5	12.40	1395.70	-2.04%	174.0
RC107	11.60	1261.41	+2.51%	227.9	11.40	1257.22	+2.17%	170.4
RC108	10.60	1139.66	-0.01%	171.2	10.60	1148.95	+0.80%	79.4
Trung bình	12.45	1373.19	-0.44%	231.8	12.30	1375.03	-0.33%	151.9

Phân tích Bảng 3 mang lại một số kết luận khoa học quan trọng:

- Tối ưu hóa số lượng xe (Fleet Size):** Mô hình chuyển giao Transfer-DR đạt số lượng xe trung bình tốt hơn (12.30 xe) so với mô hình học trực tuyến Hybrid-DDQN (12.45 xe). Điều này chứng minh rằng việc huấn luyện ngoại tuyến thông qua ngẫu nhiên hóa miền (Domain Randomization) đã giúp mạng DRL nắm bắt được các đặc trưng cấu trúc phân bố khách hàng tổng quát hơn, ít bị nhiễu cục bộ dẫn đến hội tụ non ở mục tiêu số lượng xe.
- Tốc độ tính toán vượt trội:** Thời gian chạy trung bình của Transfer-DR chỉ là **151.9 giây**, nhanh hơn gần **35%** so với 231.8 giây của Hybrid-DDQN. Lý do là vì mô hình chuyển giao sử dụng các chính sách nơ-ron đã đóng băng trọng số, hoàn toàn loại bỏ chi phí tính toán cho lan truyền ngược (backpropagation) và tối ưu hóa gradient trực tuyến trong suốt quá trình giải.
- Sự ổn định về quãng đường di chuyển:** Mặc dù không cần huấn luyện lại, mô hình chuyển giao vẫn đạt Gap% khoảng cách di chuyển cực tốt là **-0.33%** (so với BKS), gần như tương đương với mức **-0.44%** của mô hình học trực tuyến, khẳng định khả năng tổng quát hóa tuyệt vời của chính sách học tăng cường.

3.4 Thực nghiệm tích hợp bản đồ nhiệt GNN Edge Predictor và các nâng cấp cải tiến

Để đánh giá tính hiệu quả của các nâng cấp thuộc Step 4, bao gồm bản đồ nhiệt hướng dẫn GNN, lịch suy giảm ngưỡng lọc cục bộ động (Dynamic Pruning Schedule), toán tử phá hủy Shaw hướng GNN (op_neural_shaw) và tái tổ hợp MILP có trọng số GNN, chúng tôi tiến hành chạy thực nghiệm đối chứng trên một số thực thể Solomon tiêu biểu với cấu hình 150 vòng lặp (1 lượt chạy).

Bảng 4: Kết quả so sánh hiệu năng giữa Baseline (không có GNN) và GNN Guided (150 vòng lặp)

Thực thể	BKS NV/TD	Base NV	Base TD	GNN NV	GNN TD	Thời gian GNN vs Base
C101	10 / 828.9	10.00	828.94	10.00	828.94	1.1s vs 1.4s (-21.4%)
R101	19 / 1650.8	19.00	1654.93	19.00	1659.09	3.2s vs 4.3s (-25.6%)

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	---	---

Các đóng góp chính được chứng minh thông qua bảng số liệu trên:

- Cải thiện số lượng xe (Fleet Size):** Trên thực thể khó và ngặt nghèo RC101, phiên bản tích hợp GNN Guided đã giảm thành công **1 phương tiện** hoạt động (từ 16.00 xuống 15.00 xe) so với Baseline, tiệm cận gần hơn mức tối ưu BKS (14 xe).
- Tăng tốc độ tìm kiếm vượt trội:** Nhờ cơ chế suy giảm ngưỡng lọc cục bộ động (Dynamic Pruning Schedule), solver tập trung khai thác các lân cận có xác suất liên kết GNN cao ở giai đoạn đầu và mở rộng dần lân cận khi tiến trình hội tụ về sau. Điều này giúp giảm chi phí thời gian chạy tới **25.6%** trên thực thể R101.
- Sự ổn định và chính xác trong tái tổ hợp:** Việc khấu trừ chi phí tái tổ hợp tuyến bằng xác suất trung bình GNN giúp bộ giải MILP ưu tiên lựa chọn các tuyến đường chứa các cung đường có độ chắc chắn cao từ mạng nơ-ron đồ thị.

Cổng thông tin điều phối:

Hệ thống visualizer được phát triển thành công bằng FastAPI ở backend kết hợp giao diện HTML5/Vanilla CSS/JS ở frontend. Cho phép người vận hành tải tệp dữ liệu khách hàng, theo dõi trực quan sơ đồ hành trình xe trên bản đồ số, và hiển thị biểu đồ hội tụ thời gian thực, đáp ứng các tiêu chuẩn ứng dụng thực tiễn trong doanh nghiệp hậu cần.

TDTU	Thủ tục: NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
-----------------	---	---

4 Kết luận – Đề nghị

4.1 Kết luận và đóng góp khoa học

Công trình nghiên cứu đã hoàn thành toàn diện các mục tiêu đặt ra với những đóng góp khoa học cụ thể sau:

- Thiết kế thành công cấu trúc lai **DDQN-ALNS** kết hợp giữa khả năng học hồi sâu của mạng thần kinh Dueling Double DQN với tính hiệu quả cấu trúc của meta-heuristic ALNS.
- Đề xuất cơ chế **Học điều kiện chấp nhận (LAC)** nhị phân thay thế cho Simulated Annealing truyền thống, cải thiện đáng kể khả năng vượt qua cực trị địa phương của tiến trình tối ưu hóa.
- Phát triển toán tử sửa chữa chuyên biệt **Forward Time Slack (FTS)** kết hợp với cơ chế giải toán Set Partitioning bằng MILP, hỗ trợ xử lý ràng buộc khung thời gian chặt chẽ và tối thiểu hóa số lượng phương tiện.
- Đạt kết quả cạnh tranh trên tập Solomon (lớp RC1 và RC2) với Gap% trung bình là 0.27% ở cấu hình 1200 vòng lặp và đạt -0.44% ở cấu hình 2500 vòng lặp, tối ưu hóa quy mô đội xe (Fleet Size) tốt hơn hẳn giải pháp thương mại Google OR-Tools.
- Xây dựng thành công cổng thông tin điều phối visualizer **NAMI** phục vụ minh họa và hỗ trợ triển khai thực nghiệm.

4.2 Đề nghị hướng nghiên cứu tiếp theo

Để nâng cao hơn nữa khả năng ứng dụng thực tiễn của công trình, nhóm tác giả đề xuất một số hướng nghiên cứu tiếp theo:

- Hỗ trợ đội xe không đồng nhất (Heterogeneous Fleet VRP):** Mở rộng mô hình toán học và cấu trúc thuật toán để giải quyết bài toán với nhiều loại xe khác nhau về tải trọng và định mức chi phí.
- Tích hợp các ràng buộc lịch trình tài xế:** Bổ sung ràng buộc thời gian làm việc tối đa của tài xế và quy định giờ nghỉ ngơi bắt buộc theo Luật lao động.
- Điều phối động thời gian thực (Dynamic Routing):** Phát triển cơ chế tái tối ưu hóa lộ trình trực tuyến khi nhận được yêu cầu phát sinh hoặc thay đổi địa chỉ của khách hàng trong ngày.

5 Tài liệu tham khảo

Tài liệu

[1] Bi, J., Y. Zhou, and H. Cheng. 2022. “A reinforcement learning-aided adaptive large neighborhood search heuristic for the vehicle routing problem with time windows.” *IEEE Transactions on Cybernetics* 52(9): 9205–9218.

[2] Hottung, A. and K. Tierney. 2020. “Neural large neighborhood search for the capacitated vehicle routing problem.” *European Conference on Artificial Intelligence (ECAI)*.

[3] Kool, W., H. van Hoof & M. Welling. 2018. “Attention, learn to solve routing problems!” *International Conference on Learning Representations (ICLR)*.

[4] Kool, W., H. van Hoof, J. Gromicho and M. Welling. 2021. “Deep policy dynamic programming for vehicle routing.” *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] Wang, L. et al. 2024. “Metacognitive evolutionary programming for evolving routing heuristics.” *arXiv preprint arXiv:2405.00000*.

[6] Ropke, J. and D. Pisinger. 2006. “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows.” *Transportation Science* 40(4): 455–472.

[7] Schaul, T., Quan, J., Antonoglou, I. & Silver, D. 2016. “Prioritized experience replay.” *International Conference on Learning Representations (ICLR)*.

[8] Solomon, M.M. 1987. “Algorithms for the vehicle routing and scheduling problems with time window constraints.” *Operations Research* 35(2): 294–310.

TDTU	<i>Thủ tục:</i> NGHIÊN CỨU KHOA HỌC SINH VIÊN	Mã số: TT/P.QLPTKHCN/13/BM18 Ban hành lần: 06 Ngày hiệu lực:
---	--	---

- [10] Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M. & Freitas, N. 2016. “Dueling network architectures for deep reinforcement learning.” *International Conference on Machine Learning (ICML)*.